

Android.bp语法-genrule和gensrcs

在CMake中，可以调用一些外部指令来生成部分编译过程需要的依赖文件或辅助文件，然而在Android.bp中，并没有找到直接的调用支持。

经过对源码搜索，找到genrule和gensrcs，可以看到同样可以实现类似的功能。

下面以[external/iptables](#)库为例，来分析使用过程。

```
#external/iptables/extensions/Android.bp
genrule {
    name: "libext_init",
    cmd: "$(location gen_init) ' ' $(locations libxt_*.c) > $(out)",
    srcs: [
        "gen_init",
        "libxt_*.c",
    ],
    out: ["initext.c"],
    exclude_srcs: [
        // Exclude some modules that are problematic to compile (types/headers)
        "libxt_TCPOPTSTRIP.c",
        "libxt_connlabel.c",
        "libxt_cgroup.c",

        "libxt_dccp.c",
        "libxt_ipvs.c",
    ],
}

gensrcs {
    name: "libext_srcs",
    tool_files: ["filter_init"],
    cmd: "$(location filter_init) $(in) > $(out)",
    output_extension: "c",
    srcs: ["libxt_*.c"],
    exclude_srcs: [
        // Exclude some modules that are problematic to compile (types/headers)
        "libxt_TCPOPTSTRIP.c",
        "libxt_connlabel.c",
        "libxt_cgroup.c",

        "libxt_dccp.c",
        "libxt_ipvs.c",
    ],
}

cc_library_static {
    name: "libext",
    defaults: ["libext_defaults"],
    srcs: [
        ":libext_init",
        ":libext_srcs",
    ],
}
```

首先来分析genrule部分：

```

genrule {
    name: "libxt_init",
    cmd: "$(location gen_init) ' ' $(locations libxt_*.c) > $(out)",
    srcs: [
        "gen_init",
        "libxt_*.c",
    ],
    out: ["initext.c"],
    exclude_srcs: [
        // Exclude some modules that are problematic to compile (types/headers)
        "libxt_TCPOPTSTRIP.c",
        "libxt_connlabel.c",
        "libxt_cgroup.c",

        "libxt_dccp.c",
        "libxt_ipvs.c",
    ],
}

```

其中有如下几个属性：

name：模块名称，字符串类型，在所有模块中唯一。

cmd：要在一个或多个输入文件上运行的命令。Cmd支持一系列变量替换。可用的替换变量如下：

```

$(location): tools or tool_files
$(location <label>): "lable"tool, tool_file, inputoutput
$(in):
$(out):
$(depfile): depfiletrue
$(genDir): $(out)
$$: $

```

srcs：输入文件列表，字符串列表类型。

out：被生成的文件名列表，字符串列表类型。

exclude_srcs：被排除的输入文件列表，字符串列表类型。

对照前面属性介绍，我们可以了解到：

上面配置代码定义了一个名为libxt_init的genrule模块

它的输入文件有两组，第一组就是名为gen_init的单个文件，第二组就是以libxt_开头的所有.c文件

而输入文件中又将libxt_TCPOPTSTRIP.c、libxt_connlabel.c、 libxt_cgroup.c、 libxt_dccp.c和libxt_ipvs.c排除掉

定义了一个名为initext.c的输出文件

定义了一个指令，指令通过location指定指令文件gen_init完整路径，并传入了两个参数，第一个参数为空（""），第二个为通过locations指定的输入文件的完整路径，并将运行结果定向到输出文件\$(out)

指令文件gen_init内容如下：

```
#!/bin/bash -e
#
# Generate init_extensions* functions to call all the _init functions from
# filter_init
#
# Usage: gen_init <suffix> filename...
#
# Example output:
#
# void libxt_tcp_init(void);
# void libxt_udp_init(void);
# void init_extensions(void);
# void init_extensions(void) {
#     libxt_tcp_init();
#     libxt_udp_init();
# }

EXT=$1
shift

for i in "$@"; do //
    f=${i##*/}
    f=${f%.*}
    echo "void ${f}_init(void);"
done

echo "void init_extensions${EXT}(void);"
echo "void init_extensions${EXT}(void) {"

for i in "$@"; do //
    f=${i##*/}
    f=${f%.*}
    echo "    ${f}_init();"
done

echo "}"
```

通过对脚本分析，脚本主体功能就是，通过libxt_开头的文件名列表，来依次生成对应的_init函数，并依次添加到init_extensions函数调用中。

以libxt_udp.c为例，就是：

先将其完整路径中的文件名取出（f=\${i##*/}），然后将扩展名去除（f=\${f%.*}），这时保留的就是不带扩展名的文件名（libxt_udp）

然后输出“void libxt_udp_init(void)”也就是“（void \${f}_init(void)”，f将替换为libxt_udp）

输出：

```
void libxt_udp_init(void);
void init_extensions(void);
void init_extensions(void) {
    ...
    libxt_udp_init();
}
```

“void init_extensions(void)”也就是“void init_extensions\${EXT}(void);”，这里EXT也就是传入指令的第一个参数，通过前面分析，就是空参数”

其余输出以此类推。

接下来，分析gensrcs部分：

```
gensrcs {
    name: "libext_srcs",
    tool_files: ["filter_init"],
    cmd: "$(location filter_init) $(in) > $(out)",
    output_extension: "c",
    srcs: ["libxt_*.c"],
    exclude_srcs: [
        // Exclude some modules that are problematic to compile (types/headers)
        "libxt_TCPOPTSTRIP.c",
        "libxt_connlabel.c",
        "libxt_cgroup.c",

        "libxt_dccp.c",
        "libxt_ipvs.c",
    ],
}
```

其中，有如下几个属性：

name: 模块名称，字符串类型，在所有模块中唯一。

cmd: 要在一个或多个输入文件上运行的命令。Cmd支持一系列变量替换。可用的替换变量如下：

```
$(location): tools or tool_files
$(location <label>): "lable"tool, tool_file, inputoutput
$(in):
$(out):
$(depfile): depfiletrue
$(genDir): $(out)
$$: $
```

tool_files: (tool)

srcs: 输入文件列表，字符串列表类型。

output_extension: 每个输出文件被替换的扩展名，字符串类型。

exclude_srcs: 被排除的输入文件列表，字符串列表类型。

对照前面属性介绍，我们可以了解到：

上面配置代码定义了一个名为libext_srcs的gensrcs模块

它的输入文件是一组以libxt_开头的所有.c文件

而输入文件中又将libxt_TCPOPTSTRIP.c、libxt_connlabel.c、libxt_cgroup.c、libxt_dccp.c和libxt_ipvs.c排除掉

定义了一个名为c的输出文件扩展名

定义了一个指令，指令通过location指定指令文件filter_init完整路径，并传入了一个参数，即通过locations指定的输入文件的完整路径，并将运行结果定向到输出文件\$(out)

指令文件filter_init内容如下：

```
#!/bin/sh
# This is for working around Android.mk's incapability to handle $* in CFLAGS,
# even with SECONDEXPNASION.
# LOCAL_CFLAGS:=-D_INIT=$*_init
f=${1##*/}
f=${f%.*}
sed "s/\([ ]*\)\(_init\)\(\([ ]*void\)/\1${f}_init\3/" $1
```

通过对脚本分析，脚本主体功能就是，通过libtx_开头的文件名列表，来依次重命名中对应的_init函数。

以libxt_udp.c为例，就是：

先将其完整路径中的文件名取出（f=\${i##*/}），然后将扩展名去除（f=\${f%.*}），这时保留的就是不带扩展名的文件名（libxt_udp）

然后将文件（\$1）中，以“_init(void)”命名的函数，也就是“\([]*\)\(_init\)\(\([]*void\)”匹配出来的内容，将替换为libxt_udp_init(void)，并将结果最终输出到文件，在源码目录中，我们可以看到编译前后源码的变化：

```
external/iptables/extensions$ git diff libxt_udp.c
diff --git a/extensions/libxt_udp.c b/extensions/libxt_udp.c
index 0c7a4bc..c718b92 100644
--- a/extensions/libxt_udp.c
+++ b/extensions/libxt_udp.c
@@ -206,7 +206,7 @@ static struct xtables_match udp_match = {
 };

 void
-_init(void)
+_libxt_udp_init(void)
{
    xtables_register_match(&udp_match);
}
```

其中，我们可以看到，工具（filter_init脚本）在编译过程中，将文件中原始的“_init(void)”部分确实替换成了“libxt_udp_init(void)”

上述两个模块，都在libext模块中被作为输入的源文件来使用：

```
cc_library_static {
    name: "libext",
    defaults: ["libext_defaults"],
    srcs: [
        ":libext_init",
        ":libext_srcs",
    ],
}
```

最终，会被编译成名为libext的静态库。

编译过程，上面两种模块，最终都会在编译结果的out目录中生成对应的文件：

genrule:

```
$ cat out/soong/.intermediates/external/iptables/extensions/libext_init/gen/initext.c
void libxt_AUDIT_init(void);
...
void libxt_udp_init(void);
void init_extensions(void);
void init_extensions(void) {
    libxt_AUDIT_init();
    ...
    libxt_udp_init();
}
```

gensrcs:

```
$ ls out/soong/.intermediates/external/iptables/extensions/libxt_srcs/gen/gensrcs/external/iptables/extensions/
libxt_addrtype.c libxt_connbytes.c libxt_CT.c libxt_hashlimit.c libxt_length.c libxt_NFLOG.
c libxt_quota2.c libxt_SECMARK.c libxt_SYNPROXY.c libxt_TOS.c
libxt_AUDIT.c libxt_connlimit.c libxt_devgroup.c libxt_helper.c libxt_limit.c
libxt_NFQUEUE.c libxt_quota.c libxt_set.c libxt_tcp.c libxt_TPROXY.c
libxt_bpf.c libxt_connmark.c libxt_dscp.c libxt_HMARK.c libxt_mac.c libxt_osf.
c libxt_rateest.c libxt_SET.c libxt_tcpmss.c libxt_TRACE.c
libxt_CHECKSUM.c libxt_CONNMARK.c libxt_DSCP.c libxt_IDLETIMER.c libxt_mark.c libxt_owner.
c libxt_RATEEST.c libxt_socket.c libxt_TCPMSS.c libxt_u32.c
libxt_CLASSIFY.c libxt_CONNSECMARK.c libxt_ecn.c libxt_ipcomp.c libxt_MARK.c
libxt_physdev.c libxt_recent.c libxt_standard.c libxt_TEE.c libxt_udp.c
libxt_cluster.c libxt_conntrack.c libxt_esp.c libxt_iprange.c libxt_multiport.c
libxt_pkttype.c libxt_rpfILTER.c libxt_statistic.c libxt_time.c
libxt_comment.c libxt_cpu.c libxt_HARDIDLETIMER.c libxt_LED.c libxt_nfacct.c
libxt_policy.c libxt_sctp.c libxt_string.c libxt_tos.c
```

文件中内容通过比较，和替换后的原始文件相同：

```
$ md5sum external/iptables/extensions/libxt_udp.c out/soong/.intermediates/external/iptables/extensions
/libxt_srcs/gen/gensrcs/external/iptables/extensions/libxt_udp.c
bd53e2281d8bb063854f8985bd931ce1 external/iptables/extensions/libxt_udp.c
bd53e2281d8bb063854f8985bd931ce1 out/soong/.intermediates/external/iptables/extensions/libxt_srcs/gen/gensrcs
/external/iptables/extensions/libxt_udp.c
```

有关soong build更多语法描述见：

[soong_build](#)